ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/374779245

Planning and Learning: Path-Planning for Autonomous Vehicles, a Review of the Literature

Article · October 2023

CITATIONS
0

READS
34

4 authors:



Kevin Osanlou
Paris Dauphine University
11 PUBLICATIONS **37** CITATIONS
SEE PROFILE



Christophe Guettier
SAFRAN GROUP
57 PUBLICATIONS **229** CITATIONS
SEE PROFILE



Tristan Cazenave
Paris Dauphine University
251 PUBLICATIONS **2,000** CITATIONS
SEE PROFILE



Eric Jacopin
Hawkswell Studio
59 PUBLICATIONS **530** CITATIONS
SEE PROFILE

All content following this page was uploaded by Kevin Osanlou on 17 October 2023.

The user has requested enhancement of the downloaded file.

Planning and Learning: Path-Planning for Autonomous Vehicles, a Review of the Literature

Kevin Osanlou, Christophe Guettier, Tristan Cazenave, Eric Jacopin

Safran, kevin.osanlou@safrangroup.com
Safran, christophe.guettier@safrangroup.com
Paris-Dauhine University, tristan.cazenave@lamsade.dauphine.fr
Ecoles Saint-Cyr, eric.jacopin@st-cyr.terre-net.defense.gouv.fr

Abstract

This short review aims to make the reader familiar with state-of-the-art works relating to planning, scheduling and learning. First, we study state-of-the-art planning algorithms. We give a brief introduction of neural networks. Then we explore in more detail *graph neural networks*, a recent variant of neural networks suited for processing graph-structured inputs. We describe briefly the concept of reinforcement learning algorithms and some approaches designed to date. Next, we study some successful approaches combining neural networks for path-planning. Lastly, we focus on temporal planning problems with uncertainty.

Planning

The aim of planning is to conceive plans in order to achieve a particular goal. Those plans represent a sequence of actions executed by an agent that enable the transition from a *start state* of an environment, where goal requirements are not satisfied, to an *end state* where they are. In some planning tasks, states are fully observable, in others, only partially. Actions taken by the agent can be deterministic (*i.e.* lead to a certain future state) or non-deterministic (*i.e.* lead to different future states based on some probabilities that are either known or not). State variables can be continuous or not, resulting in a possibly finite or infinite number of states. Actions can be taken in parallel or only one at a time, and have a duration or not. There can be several initial start states or only one. There can be several agents or only one. Planning environments can be diverse, varying from simple positioning in a graph to the complex dynamics of a *first person shooter* (FPS) video game.

In classical planning, models are restricted in the following aspects. The environment is fully observable, there is a single agent, states are finite, there is only one known initial start state, actions are instantaneous and deterministic: there is no uncontrollable event. Actions can only be taken one at a time. Therefore, a sequence of actions from a start state will accurately define the end state, which needs to satisfy goal requirements. Generally, classical planning can be represented mathematically by a set (S, A, P) where:

- S is the set of states
- A is the set of actions
- P is a state transition function

The state transition function $P : S \times A \longrightarrow 2^S$ defines a transition from a current state $s \in S$ to another state $s' \in S$ by considering an action $a \in A$.

To express and solve planning tasks in computer science, different languages have been proposed. Each language represents components of the planning environment differently. These include the Stanford Research Institute Problem Solver (STRIPS) (Fikes and Nilsson 1971) from SRI International and the popular Planning Domain Definition Language (PDDL) (McDermott et al. 1998).

Applications of Planning in Autonomous Systems

A system is considered autonomous if it is able to generate and execute plans to achieve its assigned goals without human intervention, and if it is able to deal with unexpected events. Planning has benefited autonomous systems greatly in the past 50 years. Early on, Shakey the robot (Nilsson 1984), the first general-purpose autonomous mobile robot, was a project that saw the rise of a powerful planning algorithm known as A* (introduced in the next sections), still used nowadays. Space exploration has benefited greatly from planning techniques. Autonomy in satellites or other space vehicles reduces the need of human presence as well as communication to ground, which can be especially useful for long term missions. Applications include Deep Space 1 (Muscettola et al. 1998), or more recently the Curiosity rover (Rabideau and Benowitz 2017) which is currently exploring Mars. Aerospace applications include Unmanned Aerial Vehicles (UAVs). UAVs can be used for operational situations such as search and rescue tasks. Search and rescue operations are typically very costly both in terms of costs and human resources, and can present human risks. UAVs reduce those costs and their ability to fly autonomously allows to remove human presence for dangerous tasks. Autonomous Unmanned Ground Vehicles (AUGV) and Autonomous Ground Vehicles (AGV) are also at the center of automation efforts where (trajectory) planning is playing a crucial role. Among AGVs, self-driving cars have been the main focus for civil applications given the potentially revolutionary impact they can have on society. The most advanced self-driving cars combine the latest

Copyright © 2023, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

sensors and computer vision tools for environment perception and use planning to make relevant decisions. We refer the reader to (Badue et al. 2020) for a complete survey on self-driving cars. AUGVs on the other hand are intended for other tasks such as typical disaster relief situations, in which they can be required to perform technical actions (*e.g.* observations, measurements, communications, etc...) while navigating mostly in off-road environments across defined trajectories (Guettier and Lucas 2016). Automation allows AUGVs to perform dangerous tasks without human presence, and AGVs to move on their own while passengers can focus on other activities.

Motion Planning and Path-Planning

Path-planning consists of finding a path leading to a desired point from a start point. Motion planning consists of determining motion and path decisions for an agent in order to allow it to achieve a specified motion-related task. Motion planning is more general than path-planning in the sense that, in addition to determining a path the agent needs to take to reach an end point from a start point, it also requires motion characteristics for the agent to reach the end point. Such characteristics can be, but are not limited to, a sequence of positions over time, acceleration values to provide in order to reach a potentially required speed or parameters such as directional angles. Figure 1 illustrates the example of a motion planning task in which a robot manipulator is tasked with grabbing an object located at a START position and moving it to the GOAL position (Kumar 2020). The robot has 4 joints which can revolve. The last joint is used to grab and release objects. Let $\alpha_1, \alpha_2, \alpha_3, \alpha_4$ be the angles for each joint, starting from the base of the robot. A planning state is defined by a vector $s = (\alpha_1, \alpha_2, \alpha_3, \alpha_4)$ which entirely defines the position of the robot and the potential object it is carrying. The configuration space S is made of all possible combinations of values each α_i can take. Some states are 'legal', *i.e.* the robot can actually be in those states, others 'illegal', *i.e.* the robot cannot be in those states. For instance, supposing the angle axis is horizontal and revolves counterclockwise, any state written as $(\frac{3}{2}\pi, \alpha_2, \alpha_3, \alpha_4)$ will not be valid regardless of the values of $\alpha_2, \alpha_3, \alpha_4$ since the first joint cannot bend the joined arm downward. By discretizing values of each angle, the robot can determine a series of consecutive angle changes for each joint, which will be considered as actions, that will allow it to grab and move the object from START to GOAL.

Figure 2 shows a path-planning problem in which an agent has to move from a start grid to an end grid. In this environment, black grids represent obstacles. At each step, the agent can move to the adjacent top, left, right or bottom grids. A possible path, in red, allows the agent to fulfill its goal, and minimizes the total distance it needs to travel. The environment in Figure 2 can be represented by a geographical graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. In this graph, nodes in $\mathcal{V}$ are grids not blocked by obstacles, and edges in $\mathcal{E}$ link adjacent (non-diagonal) grids. Edges are assigned a default weight of 1 as we suppose adjacent grids to be equidistant from one another.

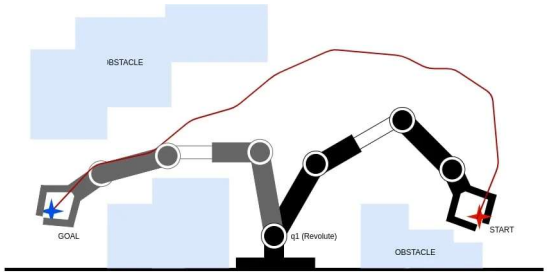


Figure 1: **A motion planning task for a robot manipulator.** The robot has to carry an object from the START location to the GOAL position. Source: (Kumar 2020)

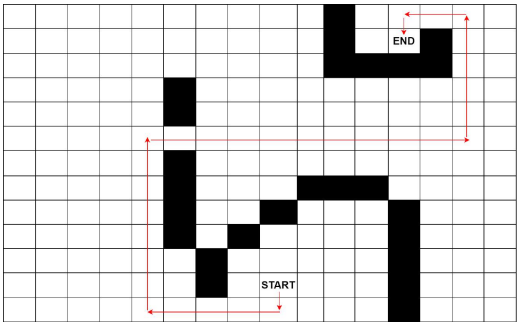


Figure 2: **A path-planning task for an agent.** Black grids represent obstacles. The agent is located at the START grid and needs to move to the END grid. The red arrows represent a possible path to satisfy that goal.

Next, we describe some popular deterministic heuristic-based algorithms for path-planning related problems. These algorithms are well-suited for planning domains with low dimensionality.

A*

The A* algorithm (Hart, Nilsson, and Raphael 1968) is a popular best-first search approach to compute an optimal path. Note that A* and related algorithms remain applicable more broadly in other planning domains than path-planning, which is what makes them so popular. A* can be considered as a specialized form of *Dynamic Programming* (DP) (Bellman 1966). DP essentially breaks down a problem into sub-problems in a recursive fashion and seeks to find the optimal choice to make at each step. This can be expressed as a search tree, which a DP algorithm will explore entirely to return an optimal solution. On the other hand, A* differs in that it will guide search towards most promising states first in order to potentially save a significant amount of computation. In path-planning problems, states are graph nodes and transition cost from a state to another is the cost of the edge linking the corresponding nodes in the graph. A* is complete: it will always find a solution if one exists in a finite search space. Depending on requirements, a heuristic which guarantees to find an optimal solution can be used, or a heuristic which simply aims to find a good solution very efficiently, even if possibly sub-optimal. We describe

this process next.

Let S be the finite set of states A* explores, s_{start} the start state where the agent starts and s_{end} the state the agent wants to transition into to satisfy goal requirements. In order to guide search, A* proceeds in a best-first fashion by keeping track, for any state s it explores, of an estimate cost $g(s)$ it took to reach that state from the start state s_{start}. Algorithm initialization is as follows: $\forall s \in S, \quad g(s) \leftarrow \infty$ and $g(s_{start}) \leftarrow 0$. Additionally, A* uses a heuristic h which estimates the remaining best cost from any state s to the goal state s_{goal}. The heuristic can be *admissible* to ensure that A* will return an optimal path: once the goal is reached, the path found is guaranteed to be optimal. The fact that the heuristic is admissible means that for any state s, $h(s)$ is lower than or equal to the actual cost of the optimal path from s to s_{goal}. A* maintains a priority queue, the $OPEN$ list, in which it inserts states by their $'f'$ value. For any state $s \in S$, $f(s) = g(s) + h(s)$. It then proceeds to extract the state s_{min} with the lowest such value in the $OPEN$ list. A* then develops all neighboring states $s' \in S' \subset S$ it can transition into from the state s_{min}. For each of those states, costs are updated if possible. More specifically, $\forall s' \in S'$, if $g(s_{min}) + TC(s_{min}, s') < g(s')$ then $g(s') \leftarrow g(s_{min}) + TC(s_{min}, s')$. Here, $TC(s, s')$ returns the transition cost from state s to state s'. Additionally, if $g(s')$ is updated, the state s' is added to the $OPEN$ list with its new f value (or its f value is updated if already present in the $OPEN$ list). The best predecessor state for s' is also stored in memory if $g(s')$ is updated, *i.e.* $prev(s') \leftarrow s_{min}$, where function $prev$ stores a predecessor for each state. The A* algorithm will keep extracting states from the $OPEN$ list until the goal state s_{goal} is extracted, at which point a path has been found (and is optimal if an admissible heuristic is used) from s_{start} to s_{goal}.

Incremental Planning

In some scenarios, the agent might not have accurate information about graph structure. The agent may acquire more accurate information about graph structure only when it has started travelling on a computed plan. This is also the case for autonomous vehicles agents if the explored terrain, represented by a graph, is inaccurate at the time of path-planning. It may also be the case if terrain structure changes are likely to happen frequently. The agent will only be able to take into account corrections as it is exploring the terrain.

If the agent computes a path from s_{start} to s_{goal}, proceeds on the path, and observes graph changes along the way (*e.g.* edge connection or weight modifications), the computed path may turn out to be in fact sub-optimal after taking into account the new graph structure. In order to compute the new optimal path from the agent's current position s on the path (when the change is observed) to s_{goal}, two possibilities exist. The agent can re-plan from scratch in order to compute the shortest path from s to s_{goal}. This approach can however cause expensive computations that may be avoidable (*e.g.* if the change in the graph does not change the optimality of the shortest path, or if it's a minor change that can be fixed with a small modification). The other possibility is to leverage information from the previously computed shortest path in order to repair it and make it optimal again. This is the approach taken in *incremental* planning algorithms. Two particularly popular such algorithms are the D* algorithm (Stentz et al. 1995) and an improved lighter version of D*, the D* Lite algorithm (Koenig and Likhachev 2002a). D* Lite is quite efficient and remains a method of choice even now, with a wide array of applications relying on it.

Simply put, D* and D* Lite aim to re-expand and develop only parts of the search space relevant to registered graph changes and the potential new current state the agent is in. The following provides an overview of how D* Lite operates. First, it computes a path from s_{start} to s_{goal} using backwards A*. Backwards A* works in the same way as A* except the search is done backwards: from the goal state s_{goal} to the start state s_{start}. Furthermore, a consistency criterion is used for each state s explored. This criterion compares the cost of the optimal path found from state s to s_{goal} to the minimum of the costs to s_{goal} obtained from each neighboring state plus the transition cost to said neighboring states. The state is said to be consistent if they are equal. Otherwise, it is said to be inconsistent (either overconsistent or underconsistent if respectively higher than or lower than). When a change is observed in the graph while the agent is proceeding on the computed shortest path, edges are updated, and the resulting inconsistent states are re-processed in a defined priority order. Once the process is over, the path has been repaired and is optimal again.

Sometimes changes observed which would result in no impact on the optimality of the path will still require computations by D* and D* Lite to guarantee optimality. Such is the case if some edge weights, all of which are outside the computed path, increase. The algorithm would still need to reprocess states becoming inconsistent due to their connection to edge changes before guaranteeing optimality, even though it is clear the path computed is still optimal. To address this issue, a modified version of D*, delayed D* (Ferguson and Stentz 2005), has been proposed. To avoid useless computations in such situations, delayed D* initially ignores underconsistent states and only focuses on overconsistent states first. This enables it to potentially save a lot of computations in such cases, making it more suited than D* Lite in some planning domains (Ferguson and Stentz 2005).

Another incremental approach worthy of note is Lifelong Planning A* (LPA*) (Koenig and Likhachev 2002b). LPA* starts by running an A* instance to determine an optimal path from a start state s_{start} to a goal state s_{goal}. Once edge changes are observed, it uses previous search information to re-compute an optimal path more efficiently in a similar way to D*. The main difference with D* is that LPA* does not allow s_{start} and s_{goal} to be modified. In other words, the approach can only be used before the agent starts moving on the path, in case some last-minute changes are learned (presumably remotely). It is thus unsuitable in situations where the agent observes changes as it is already moving on a computed path and needs to adjust the plan from a new position. More recently, (Przybylski and Putz 2017) proposed the D* Extra Lite algorithm. Similarly to D* Lite, D* Extra Lite is based on A* and propagates changes to the previously processed search space in order to re-optimize a path. Unlike

D* Lite, the reinitialization of the affected search space is achieved by cutting search tree branches. This allows the algorithm to often outperform D* Lite, with experiments suggesting it can be almost up to twice faster on typical path-planning problems.

Previously described approaches are applicable in graphs, and are therefore well-suited to, for example, grid environments where agents can move with 45 or 90 degree angles. Such a representation of the environment can cause the optimal path in the graph to actually be sub-optimal in reality. In *any angle* path-planning, the agent can take any angle to move around in its environment. Some incremental planning work have also emerged for such environments. (Ferguson and Stentz 2007) introduce Field D*, an adaptation of the D* algorithm for any angle path-planning, which reportedly returns a solution path often close to the optimal solution. Other works include Theta* from (Nash et al. 2007). Based on A*, Theta* is shown to give even shorter paths than Field D*, though not necessarily optimal either. However, Theta* lacks Field D*'s fast replanning capabilities. Finally, (Harabor et al. 2016) introduced ANYA, which they show to be significantly faster than previous approaches. Moreover, ANYA also guarantees to find optimal any-angle paths.

Anytime Planning

In some situations a path needs to be computed quickly. Such could be the case for example for an agent detecting possible obstruction on a planned path while in movement. A solution would be required by the agent as fast as possible to avoid having to come to a complete stop and waste time while re-computing a path. Computing a new optimal path can quickly become very hard, even for incremental algorithms if the number of search states required to be reprocessed is high. In such a situation, it can be acceptable to compute a solution which is not guaranteed to be optimal very quickly first, so that the agent can keep moving. In the remaining time available (*e.g.* time for the agent to reach decisive points), the previously computed (likely sub-optimal) path can be improved. *Anytime* algorithms, sometimes referred to as *Hierarchical* path-planners, are designed to address that problem. They build a likely sub-optimal path very quickly and improve the path in the remaining time available.

There have been various works on anytime algorithms. (Likhachev, Gordon, and Thrun 2003) introduced the well-known Anytime Repairing A* (ARA*). This algorithm is made of successive weighted A* searches. In a weighted A* search, the heuristic function h used is multiplied by a factor $\epsilon > 1$. In doing so, substantial speedup is often provided at the cost of solution optimality. ARA* executes successive weighted A* searches with a decreasing inflation factor ϵ, each of which uses information from previous searches and provides a sub-optimality bound. During each weighted A* search, ARA* considers only states whose costs at the previous search may not be valid anymore due to the new, lower ϵ value. Another anytime algorithm is the Anytime Weighted A* (AWA*) (Hansen and Zhou 2007), which is very similar to ARA*. Authors show that AWA* is seven times faster than ARA* on certain domains such as the sliding-tile planning problem of eight puzzles.

From another perspective, (Likhachev et al. 2005) introduced Anytime Dynamic A* (AD*). Unlike previous approaches, AD* does not differentiate incremental and anytime approaches. Instead, it provides a framework which combines the benefits of both to provide solutions efficiently to hard dynamic problems. Experiments are carried out in an environment where a robotic arm is manipulating an end-effector through a dynamic environment and show AD* generating significantly better trajectories than ARA* and D* Lite in the same time budget. (Botea, Müller, and Schaeffer 2004) presented Hierarchical Path-Finding A* (HPA*). HPA* proceeds to divide the environment into square clusters with connections, making an abstract search graph which is searched to find a shortest path. Another approach, Partial-Refinement A* (PRA*) (Sturtevant and Buro 2005), builds cliques of nodes to construct a multi-level search space. The original problem is reduced to finding a set of nodes on the optimal shortest path. However, both HPA* and PRA* address homogenous agents in homogenous-terrain environments. An extension of HPA*, Annotated Hierarchical A* (AHA*), has been proposed by (Harabor and Botea 2008). It is still one of the most advanced anytime path-planning algorithms to date. AHA* is able to deal with heterogeneous multi-terrain environments by reducing them to simpler single-size, single-terrain search problems. Authors' experiments suggest that near-optimal solutions are returned by the algorithm for problems in a wide range of environments, with an exponentially lower search effort than A*.

Probabilistic Methods for Path-Planning

In high-dimensional search spaces, probabilistic approaches can provide a solution quickly but not necessarily an optimal one. We describe two popular approaches, Probabilistic Roadmaps (PRM) (Kavraki et al. 1996) and Rapidly-exploring Random Trees (RRT) (LaValle 1998). The intuition behind PRMs is to generate random 'points' in the search space, connect these points to nearby points, and repeat the procedure until a path can be computed from the start state s_{start} to the goal state s_{goal} by moving along these points. More specifically, PRM starts by generating random states. It checks whether the generated states are valid, *i.e.* if they do not possess contradictory features (*e.g.* for the robot manipulator in 1, one would need to check if the combination of angles does not leave the robot arm in an impossible position). Invalid states are removed, and remaining states are named "milestones". Each milestone is connected to its k-nearest neighbor states, k being a parameter. The process is repeated until the roadmap (the milestones and their connections) becomes dense enough and a connection between s_{start} and s_{goal} is created. A shortest path on the roadmap is then computed between s_{start} and s_{goal}. PRM is *probabilistically complete*, *i.e.* as the roadmap building process goes on in time, the probability that the algorithm will find an existing path from s_{start} to s_{goal} tends to 1. Figure 3 illustrates a PRM. Notable follow-up works include Hierarchical PRMs (Collins, Agarwal, and Harer 2003), which are a variant of PRMs refined recursively, providing better performance at finding narrow passages than uniform sam-

pling. Other works have attempted to improve the efficiency of PRMs by altering the state sample generation process. Recently, (Kannan et al. 2016) built a PRM variant with adaptive sampling. They assign probabilities to different samplers dynamically based on the environment and use the one with the highest probability. (Ichter et al. 2020) proposed to learn to identify 'critical' states with a neural network from local environment features, *i.e.* states that are key to building the wanted path (*e.g.* doorways in an office environment). They draw these critical samples more often and thus are able to build a hierarchical roadmap more efficiently, with reportedly up to three order of magnitude improvements in computation time.

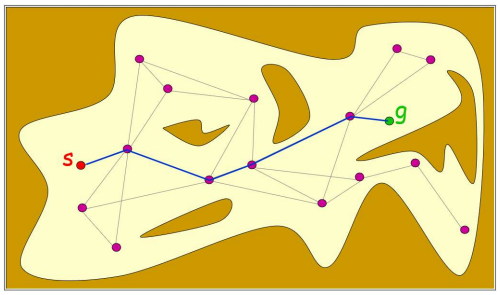


Figure 3: **A Probabilistic roadmap.** The white space represents feasible states, purple points milestones. Point 's' is the start state, point 'g' the goal state. The shortest path on the roadmap linking s to g is shown in blue. Credit for the picture goes to Jean-Claude Latombe. Source: (Latombe 2020)

An RRT, on the other hand, starts growing a tree rapidly from the start state s_{start}, which is considered to be its root. To do so, RRT repeatedly uses a randomly sampled state s_{rand} and attempts to connect s_{rand} to any nearest state in the tree via feasible paths. When successful, RRT expands the size of the tree further with the addition of s_{rand} and intermediary states found on the path. The sampling of random states is done in a way which expands the tree towards unsearched areas of the search space. Furthermore, for a randomly generated state s_{rand} the length of its connection to the tree is limited by a growth factor. If the total length of the connection is above this distance, s_{rand} is dropped and s'_{rand}, the state at the maximally allowed distance from the tree along the connection is selected instead. In this manner, the position of the randomly generated samples determines towards which areas the tree gets expanded, while the growth factor limits how far the tree is expanded in those directions. A drawback of RRTs is that they tend to often converge to non-optimal solutions. To address this issue, (Karaman and Frazzoli 2010) introduced RRT*, which they showed to almost surely converge towards the optimal path without any significant overhead against RRT. We describe some notable follow-up works which are variants of RRT*. (Adiyatov and Varol 2013) proposed a variant called RRT* Fixed Nodes (RRT*FN). Since there is no limit to the number of nodes RRT* can develop, the algorithm is not suited for embedded systems with limited memory. RRT*FN aims to solve the issue by using a node removal procedure which allows it to limit the number of nodes developed without hindering the convergence of the algorithm towards an optimal solution. (Gammell, Srinivasa, and Barfoot 2014) proposed informed-RRT*, a variant which uses a heuristic to shrink the planning problem to subsets of the original domain. Informed-RRT* reportedly outperforms RRT* in rate of convergence, final solution cost, and ability to find difficult passages. More recently, (Lai, Ramos, and Francis 2019) presented Rapidly-exploring Random disjointed-Trees* (RRdT). It is a RRT* variant which explores the search space with locally exploring disjointed trees and actively balances global exploration and local-connectivity exploitation. This is done by expressing the problem as a multi-armed bandit problem, and leads to improved performance.

Graph Representation Learning with Graph Neural Networks

Neural Networks

We start by giving a brief description of neural networks and convolutional neural networks, architecture types from which graph neural networks originated.

Neural Networks (NNs) allow abstraction of data by using models with trainable parameters coupled with non-linear transformations of the input data. In spite of the complex structure of a NN, the main mechanism is straightforward. A *feedforward neural network*, or *Multi-Layer Perceptron (MLP)*, with L layers describes a function $f_{\boldsymbol{\theta}}(\mathbf{x}) = f(\mathbf{x}; \boldsymbol{\theta}) : \mathbb{R}^{d_{\mathbf{x}}} \rightarrow \mathbb{R}^{d_{\hat{\mathbf{y}}}}$ that maps an input vector $\mathbf{x} \in \mathbb{R}^{d_{\mathbf{x}}}$ to an output vector $\hat{\mathbf{y}} \in \mathbb{R}^{d_{\hat{\mathbf{y}}}}$. Vector $\mathbf{x}$ is the input data that we need to analyze (*e.g.* an image, a signal, a graph, etc.), while $\hat{\mathbf{y}}$ is the expected decision from the NN (*e.g.* a class index, a heatmap, etc.). The function f performs L successive operations over the input $\mathbf{x}$:

$$h^{(l)} = f^{(l)}(h^{(l-1)}; \theta^{(l)}), \qquad l = 1, \ldots, L \tag{1}$$

where $h^{(l)}$ is the hidden state of the network (*i.e.* features from intermediate layers, corresponding to intermediary values) and $f^{(l)}(h^{(l-1)}; \theta^{(l)}) : \mathbb{R}^{d_{l-1}} \mapsto \mathbb{R}^{d_l}$ is the mapping function performed at layer l; $h_0 = \mathbf{x}$. In other words:

$$f(\mathbf{x}) = f^{(L)}(f^{(L-1)}(\ldots f^{(1)}(\mathbf{x}) \ldots))$$

Each intermediate mapping depends on the output of the previous layer and on a set of trainable parameters $\theta^{(l)}$. We denote by $\boldsymbol{\theta} = \{\theta^{(1)}, \ldots, \theta^{(L)}\}$ the entire set of parameters of the network. The intermediate functions $f^{(l)}(h^{(l-1)}; \theta^{(l)})$ have the form:

$$f^{(l)}(h^{(l-1)}; \theta^{(l)}) = \sigma\left(\theta^{(l)} h^{(l-1)} + b^{(l)}\right), \tag{2}$$

where $\theta^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$ and $b^{(l)} \in \mathbb{R}^{d_l}$ are the trainable parameters and the bias, while $\sigma(\cdot)$ is an *activation* function, *i.e.* a function which is applied individually to each element of its input vector to introduce non-linearities. Intermediate layers are actually a combination of linear classifiers followed by a piecewise non-linearity. Layers with this form are termed *fully-connected layers*.

NNs are typically trained using labeled training data from a dataset, *i.e.* a set of input-output pairs $(\mathbf{x}_i, \mathbf{y}_i)$, $i = 1, \ldots, N$, where N is the size of the dataset. During training we aim to minimize the training loss:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \ell(\hat{\mathbf{y}}_i, \mathbf{y}_i), \tag{3}$$

where $\hat{\mathbf{y}}_i = f(\mathbf{x}_i; \boldsymbol{\theta})$ is the estimation of $\mathbf{y}_i$ by the NN and $\ell : \mathbb{R}^{d_L} \times \mathbb{R}^{d_L} \mapsto \mathbb{R}$ is a loss function which measures the distance between the true label $\mathbf{y}_i$ and the estimated one $\hat{\mathbf{y}}_i$. Through *backpropagation*, the information from the loss is transmitted to all $\boldsymbol{\theta}$ and gradients of each θ_l are computed w.r.t. the loss. The optimal values of the parameters $\boldsymbol{\theta}$ are then searched for via Stochastic Gradient Descent (SGD) which updates $\boldsymbol{\theta}$ iteratively towards the minimization of $\mathcal{L}$. The input data is randomly grouped into mini-batches and parameters are updated after each pass. The entire dataset is passed through the network multiple times and the parameters are updated after each pass until reaching a satisfactory optimum. In this manner all the parameters of the NN are learned jointly and the pipeline allows the network to learn to extract features and to learn other more abstract features on top of the representations from lower layers.

In recent years, NNs, in particular Deep Neural Networks (DNNs), have achieved major breakthroughs in various areas. While the fundamental principles of training neural networks are known since many years, the recent improvements are due to a mix of availability of large datasets, advances in GPU-based computation and increased shared community effort. Similarly to NNs, DNNs enable a high number of levels of abstraction of data by using models with millions of trainable parameters coupled with non-linear transformations of the input data. It is known that a sufficiently large neural network can approximate any continuous function (Funahashi 1989), although the cost of training such a network can be prohibitive.

Convolutional Neural Networks (CNNs) (Fukushima and Miyake 1982; LeCun, Bengio et al. 1995) are a generalization of multi-layer perceptrons for 2D data. In convolutional layers, groups of parameters (which can be seen as small fully-connected layers) are slided across an input vector similarly to filters in image processing. This reduces significantly the number of parameters of the network since they are now *shared* across locations, whereas in fully connected layers there is a parameter for element of the input. Since the convolutional units act locally, the input to the network can have a variable size. A convolutional layer is also a combination of linear classifiers (equation 2) and the output of such layer is 2D and is called *feature map*. CNNs are highly popular in most recent approaches for computer vision problems. Figure 4 shows a CNN architecture.

Graph Neural Networks

In this section, we discuss various architectures of Graph Neural Networks (GNNs). GNNs are generalizations of CNNs to non-Euclidean data and aim to learn graph representations. Although no common groups have been precisely defined for GNNs, they tend to belong to four categories:

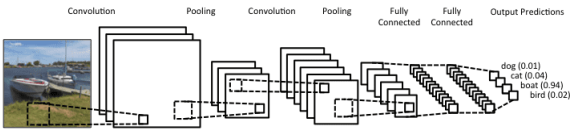


Figure 4: **A convolutional neural network.** The input of this CNN is an image, the output a prediction of a class among the following set of classes: {dog,cat,boat,bird}.

- **Converging Recurrent Graph Neural Networks (CRGNN)**. These architectures of neural networks mostly include the first works on extending NNs to graphs.
- **Graph Convolutional Networks (GCN)**. These networks are mostly inspired from the application of CNNs to graphs and are well-suited to supervised-learning for node classification.
- **Recurrent Graph Neural Networks (RGNN)**. RGNNs are designed to process input graphs for which a temporal sequence ordering exists. They should not be confused with CRGNNs, which are 'recurrent' in the sense they apply a process on the input graph repeatedly until convergence.

These networks have different processing architectures but have the following common point. They take as input a graph in which nodes, and possibly edges, have features. They use intermediary layers, each of which produces new features for nodes (and possibly edges, depending on the architecture type). Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an input graph of these network architectures with its set of nodes $\mathcal{V} = (v_1, v_2, ..., v_n)$ and its set of edges $\mathcal{E}$. We denote as A the adjacency matrix of $\mathcal{G}$, $X = (x_{v_1}, x_{v_2}, ..., x_{v_n})$ the matrix of node feature vectors of the input graph $\mathcal{G}$ and $H^{(l)} = (h_{v_1}^{(l)}, h_{v_2}^{(l)}, ..., h_{v_n}^{(l)})$ the matrix of node feature vectors after the input graph $\mathcal{G}$ has been processed by l layers. Each x_{v_i} is the feature vector of node v_i of the input graph $\mathcal{G}$ and each $h_{v_i}^{(l)}$ the feature vector of node v_i after the lth layer. Finally, as the layer architecture types we describe next apply a similar process to each graph node, the same layer can be used on input graphs with any number of nodes, although the number of features per node needs to be fixed. In other words, a same GNN with layers made of these architectures can process input graphs with any number of node.

Converging Recurrent Graph Neural Networks The idea behind CRGNNs was initially introduced in (Sperduti and Starita 1997), with a contribution termed *generalized recursive neuron*, extending the idea of applying neural networks to inputs with structures. Those structures were essentially limited to acyclic graphs because of computational constraints at the time. In follow-up works, (Scarselli et al. 2008) extend this with an architecture capable of processing acyclic, cyclic, directed and undirected graphs. To that end, neighborhood information among graph nodes is exchanged repeatedly until convergence. The following formula describes how information is updated from layer l to

layer $l+1$ for node v:

$$h_v^{(l)} = \sum_{w \in N(v)} f(x_v, x_{e_{vw}}, x_w, h_w^{(l-1)}) \quad (4)$$

where:

- $h_v^{(l)}$ and $h_v^{(l-1)}$ respectively designate the vector feature of node v after layer l and layer $l-1$; $h_v^{(0)} = x_v$.
- $N(v)$ designates the nodes connected to node v with an edge.
- $x_{e_{vw}}$ is the feature vector of the edge connecting v and w.
- f is a parametric function, called *local transition function* by Scarselli *et al.*

Intuitively, the information update of node v from a layer $l-1$ to a layer l proceeds in the following manner for each node. For each neighboring node w, a parametric function f takes as input the following elements: the input feature vectors of node v, edge (v,w), node w, as well as the feature vector of node w after layer $l-1$. The sum of the output of f for each neighbor of v makes the new feature vector of node v after layer l. Moreover, to ensure convergence after applying layers repeatedly, function f needs to be a *contraction map* which reduces the distance between inputs and satisfies this property:

$$\forall z \in \mathbb{R}^m \ \exists \mu \in]0,1[\ s.t \ \forall (x,y) \in \mathbb{R}^{m' \times m''} : \ \|f(x,z) - f(y,z)\| \leqslant \mu \|x - y\|$$

where $\|\cdot\|$ denotes a vectorial norm.

A convergence criterion also needs to be defined. Layers are applied recursively on each node in parallel until this criterion is satisfied. The converged node feature vectors $h_{v_i}^*$ of each node v_i can then be forwarded to an output layer to perform either node classification tasks, edge classification tasks (by using for example a MLP which takes as input converged features $h_{v_i}^*$ and $h_{v_j}^*$ and outputs a value for edge (v_i, v_j)) or graph-level predictions (*e.g.* predict a class among a portfolio of classes for the input graph, which is typically done by using one or multiple *pooling* operations such as max or min to reduce the size of the converged graph into a fixed size, enabling the use, for example, of a fully-connected output layer).

A notable issue with this CRGNN architecture is the number of layers which need to be applied to meet the convergence criterion and the possibly ensuing complexity. More recently, a framework was proposed in (Li et al. 2015) to address this issue based on gated recurrent units (Cho et al. 2014). This allows Li *et al.* to only require a fixed number of layers to process an input graph, thereby lifting the constraints associated with the convergence criterion. Nevertheless, the approach in (Li et al. 2015) requires Back-Propagation Through Time (BPTT) to compute gradients when using the model in a loss function, which can cause severe overhead.

CRGNNs are mostly pioneer works which inspired the next architectures we describe, and even the newest CRGNN approach presents computational issues due to BPTT.

Graph Convolutional Networks Unlike CRGNNs where a fixed recurrent model is applied repeatedly, GCNs use a fixed number of graph convolutional layers, each of which is different and has its own set of trainable parameters. GCNs are inspired from CNNs. They generalize their operations from grid-structured data (images) to graph-structured data. There are two main categories of GCNs: *spectral-based* and *spatial-based*. Spectral-based approaches use signal processing to define the neighborhood of a node and the ensuing feature update process, while spatial-based approaches rely directly on spatially close neighbors in the graph.

Spectral-based GCNs Spectral-based architectures use the spectral representation of graphs and are thus limited to undirected graphs. They were introduced in (Bruna et al. 2014). The following layer propagation rule is used to compute $H^{(l)}$, the matrix of all node feature vectors at layer l, from $H^{(l-1)}$:

$$H^{(l)} = \sigma(U g_\theta(\Lambda) U^T H^{(l-1)}) \quad (5)$$

Here, U denotes the eigenvectors of the normalized graph Laplacian matrix $L = I_N - D^{-\frac{1}{2}} A D^{-\frac{1}{2}}$ (A being the adjacency matrix, D the node degree matrix) and Λ its eigenvalues. Function $g_\theta(\Lambda) = diag_\theta(\Lambda)$ is a filter applied on the eigenvalues with a set of parameters θ. Lastly, σ is an activation function. A problem with this approach is that it results in non-spatially localized filters, making it unable to extract local features independently of graph size. In a follow-up work, (Defferrard, Bresson, and Vandergheynst 2016) introduce *ChebNet*. The filters proposed in ChebNet are localized in space. Their idea is to replace $g_\theta(\Lambda)$ with a truncation of Chebyshev polynomials T_k of the eigenvalues Λ: $g_\theta(\Lambda) = \sum_{k=0}^{K} \theta_k T_k(\tilde{\Lambda})$ where $\tilde{\Lambda} = \frac{2\Lambda}{\lambda_{max}} - I_n$, $\theta_k \in \mathbb{R}^K$ and λ_{max} denotes the highest eigenvalue. Chebyshev polynomials are recursively defined by: $T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x)$; $T_0(x) = 1$; $T_1(x) = x$. After further simplifications, the layer propagation rule is simplified to:

$$H^{(l)} = \sigma(\sum_{k=0}^{K} \theta_k T_k(\tilde{L}) H^{(l-1)}) \quad (6)$$

where $\tilde{L} = \frac{2L}{\lambda_{max}} - I_n$. More recently, in (Kipf and Welling 2017), authors introduce *GCN* by applying a first-order approximation of ChebNet ($K = 1$, and $\lambda_{max} = 2$). This enables them to avoid overfitting local neighborhood structures on graphs with unbalanced node degree distributions. Equation 6 becomes:

$$H^{(l)} = \sigma(\theta_0 H^{(l-1)} - \theta_1 D^{-\frac{1}{2}} A D^{-\frac{1}{2}} H^{(l-1)}) \quad (7)$$

An additional assumption is made in GCN that $\theta = \theta_0 = -\theta_1$ to further reduce overfitting, and the equation becomes:

$$H^{(l)} = \sigma(\theta(I_n + D^{-\frac{1}{2}} A D^{-\frac{1}{2}}) H^{(l-1)}) \quad (8)$$

Finally, a *re-normalization trick* is used to avoid numerical instabilities such as exploding or vanishing gradients: $I_n + D^{-\frac{1}{2}} A D^{-\frac{1}{2}} \rightarrow \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}}$, with $\tilde{A} = A + I_n$ and $\tilde{D}$

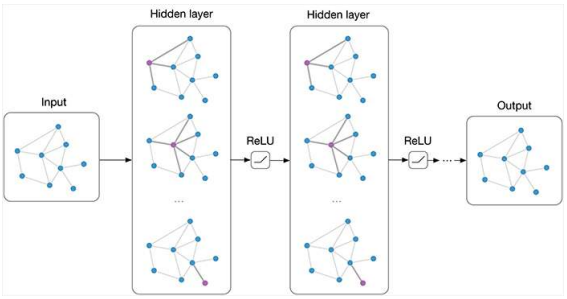


Figure 5: **A graph convolutional network.** In this illustration, an input graph with node features (and possibly edge features) is processed through multiple graph convolutional layers and $ReLU(\cdot) = max(0, \cdot)$ nonlinearities. An output graph is returned with new, updated node features. Credit goes to Thomas Kipf for the illustration, source: (Kipf 2021)

being the degree matrix of $\tilde{A}$. Kipf and Welling generalize this definition to an input $H^{(l-1)} \in \mathbb{R}^{N \times C}$ where C is the number of features per node at layer $l-1$, N the number of nodes in the input graph. Moreover, They use a weight matrix $W \in \mathbb{R}^{C \times F}$, where F is the desired number of features per node after the layer has been applied. The equation becomes:

$$H^{(l)} = \sigma(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(l-1)}W) \tag{9}$$

On a side note, during the information update for a node v, GCN takes a weighted sum of vector features from neighbors, where the weight for a neighbor w is given by: $\frac{1}{\sqrt{deg(v) \times deg(w)}}$, where $deg(v)$ refers to the degree of node v. Several linear combinations are then applied, to create as many output features as needed for v in the next layer.

Lastly, methods presented thus far rely on the adjacency matrix to define relations between nodes, possibly missing on implicit information between nodes. Authors in (Li and Wang 2018) propose Adaptive Graph Convolutional Network (AGCN) to address this issue. AGCN basically learns a *residual* graph adjacency matrix by learning a distance function which takes as input the features of two different nodes in the graph, enabling it to better capture implicit dependencies.

Spatial-based GCNs Spatial-based approaches rely on spatially close neighbors to define the feature update step for a node. In this sense, spatial-based GCNs are somewhat similar to CRGNNs in that they propagate node information through edges, although they do not retain the idea of convergence and they stack multiple different layers with different trainable weights. A significant advantage of spatial-based GCNs over spectral-based GCNs is that they can be used on directed graphs.

Among early spatial-based architectures, (Micheli 2009) introduces neural network for graphs (NN4G). In the NN4G architecture, graph convolutions are performed at each layer (each of which has its own trainable weights). Each convolution basically consists in the sum, for each node, of the feature vectors of neighboring nodes. In this sense, it is somewhat similar to the GCN architecture of (Kipf and Welling 2017) which performs a weighted sum based on the spectral graph instead. Additionally, NN4G applies residual skip connections between each layer to 'memorize' information. Each new layer is basically linked not only to the previous one, but also to all preceding layers and the input. The following equation defines NN4G's propagation rule, where $\Theta^{(l)}$ and $W^{(k)}$ are weight matrices:

$$H^{(l)} = \sigma(X\Theta^{(l)} + \sum_{k=1}^{l-1} AH^{(k)}W^{(k)}) \tag{10}$$

The Diffusion Convolutional Neural Networks (DCNNs) proposed in (Atwood and Towsley 2016) brings the concept of diffusion to graphs convolutions. A transition probability is defined when information from a node is passed to a neighboring node, causing the passing of information to converge after applying the process repeatedly. Transition matrices are used to define the neighborhood for a node. The propagation rule for DCNN is:

$$H^{(l)} = \sigma(W^{(l)} \odot P^l H^{(l-1)}) \tag{11}$$

where $W^{(l)}$ is a weight matrix, $\odot$ denotes the element-wise product, P^l (not to be confused with $P^{(l)}$) is P to the power of l, with $P = D^{-1}A$ the probability transition matrix.

Message Passing Neural Networks (MPNN), on the other hand, are a general framework presented in (Gilmer et al. 2017) which aim to regroup different categories of previous works into one single architecture. In MPNNs, during the convolution phase of an input graph, messages are passed between nodes along edges by following an aggregation phase, called *message passing* phase, after which node features get updated in a *message update* phase. Each node v has its feature vector $h_v^{(l-1)}$ updated to $h_v^{(l)}$ based on a message $m_v^{(l)}$:

$$m_v^{(l)} = \sum_{w \in N(v)} M_{l-1}(h_v^{(l-1)}, h_w^{(l-1)}, x_{e_{vw}}) \tag{12}$$

$$h_v^{(l)} = U_{l-1}(h_v^{(l-1)}, m_v^{(l)}) \tag{13}$$

where $N(v)$ designates the neighborhood of node v; M_{l-1} and U_{l-1} are learned differentiable functions; $x_{e_{vw}}$ is the vector feature of the edge connecting v and w.

A *readout phase* is also introduced (after the last message passing layer l_{max} has been applied), in which a readout function R can optionally compute a feature vector $\hat{y}$ for the whole graph (assuming we want to do some other type of classification than node classification, such as graph-level class prediction):

$$\hat{y} = R(\{h_v^{(l_{max})} | v \in \mathcal{V}\}) \tag{14}$$

R needs to be invariant to the permutation of node states in order for the MPNN to retain invariance to graph isomorphism. Gilmer *et al.* proceed to express previous existing GNN architectures in the literature by specifying the corresponding message passing function M_{l-1}, message update function U_{l-1} and readout function R. In their own

work, they use an architecture in which $M(h_v, h_w, x_{e_{vw}}) = MLP(x_{e_{vw}})h_w$. Here, MLP is a multi-layer perceptron which takes as input the feature vector of edge (v, w) and outputs a $out_c \times in_c$ sized-matrix, out_c being the number of desired feature per node after applying the message passing layer and in_c the number of feature per node of the input graph provided to the layer. Vector h_w being of size $in_c \times 1$, the matrix multiplication results in a $out_c \times 1$ sized-matrix, *i.e.* a vector which has the desired number of new features after the message passing layer is applied. The sum of these vectors for the entire neighborhood defines m_v. Gilmer *et al.* apply this architecture for node classification tasks on a molecular property prediction benchmark and achieve state-of-the-art results.

Other recent relevant works include GraphSAGE (Hamilton, Ying, and Leskovec 2017) and Graph Attention Networks (GATs) (Veličković et al. 2018). GraphSAGE has been conceived to handle graphs where the number of neighbors for nodes can vary greatly from one node to another. Since always taking into account the entire neighborhood can prove inefficient and costly, graphSAGE uses sampling to define neighborhoods and thus keep a fixed number of neighbors for each node. The propagation rule in a graphSAGE convolution is defined by:

$$h_v^{(l)} = \sigma[W^{(l)} AGG_l(\{h_v^{(l-1)}\} \cup \{h_u^{(l-1)}, \forall u \in N_r(v)\})] \tag{15}$$

where: $N_r(v)$ designates a fixed-size uniform draw from the set $\{u \in \mathcal{V} : (u, v) \in \mathcal{E}\}$ and AGG_l is an aggregation function invariant to the permutations of node orderings (*e.g.* mean function). GATs, on the other hand, use an *attention* mechanism which defines weights for each connected pair of nodes. Weights are learned by the attention mechanism so as to reflect the importance of each neighbor of a node v. The layer propagation rule is defined by:

$$h_v^{(l)} = \sigma(W^{(l)} \sum_{u \in \{v\} \cup N(v)} \alpha_{uv}^{(l)} h_u^{(l-1)}) \tag{16}$$

where $N(v)$ refers to the neighborhood of v, and $\alpha_{uv}^{(l)}$ is the attention weight.

Additionally, GAT can use multi-head attention mechanisms (*i.e.* have multiple attention heads $\alpha_{uv}^{(l)}$, $\alpha_{uv}^{(l)'}$, $\alpha_{uv}^{(l)''}$, etc...). This enables the model to learn different attention schemes in parallel per layer, and shows considerable improvement over GraphSAGE on node classification benchmarks.

Recurrent Graph Neural Networks In many applications, graphs can not only present spatial structure, but also hold temporal dependencies. An example is road network traffic, for which the same graph at different time steps is going to represent the current flow of traffic in the network. RGNNs are inspired by Recurrent Neural Networks (RNNs) and aim to process a sequence of temporal graphs, in order for example to make predictions about future states (*e.g.* how traffic is going to be like in future time steps). For most RGNNs, an RNN-like mechanism is used to memorize and leverage temporal information. Nevertheless, some RGNNs use CNNs to capture temporal information instead. We first describe some RNN-based methods and then some CNN-based approaches.

The idea behind RNN-based RGNNs stems from the recurrent units used in RNNs. When a RNN is used on an input at time step t, each hidden layer ${h^{(l)}}^t$ is computed by combining both the input to the layer ${h^{(l-1)}}^t$, as well as a 'memory' equal to the output of the same layer at time step $t-1$: ${h^{(l)}}^{t-1}$.

Main such works include Structural-RNN (S-RNN) (Jain et al. 2016). S-RNN uses different RNNs to handle both node and edge information, namely *nodeRNN* and *edgeRNN*. Diffusion Convolutional Recurrent Neural Network (DCRNN) (Li et al. 2017) is an encoder-decoder framework which applies gated recurrent units on the DCNN architecture. In (Seo et al. 2018), a Long Short-Term Memory (LSTM) network is combined with the ChebNet graph convolution operator. LSTMs are a popular type of RNN architecture because they are able to maintain a longer memory than RNNs.

CNN-based RGNNs, on the other hand, abandon the idea of keeping a memory and instead use a CNN jointly with a graph convolution operator to capture temporal and spatial information at the same time. Their advantage over RNN-based RGNNs is that they do not require backpropagation through time for gradient computation. The idea is that for each node v in the input graph, a 1D-CNN is applied and temporal information from previous states of the node is aggregated. Next, a graph convolutional layer is applied on the aggregated temporal information to aggregate spatial information. This process is repeated for each layer.

Graph WaveNet (Wu et al. 2019) introduces a framework with a self-adaptive adjacency matrix. This allows Graph WaveNet to learn latent structures, which can help discover implicit temporal dependencies between nodes in the graph. Lastly, (Guo et al. 2019) introduce an Attention based Spatial-Temporal Graph Convolutional Network (ASTGCN) to solve traffic flow forecasting problems. ASTGCN builds on STGCN by introducing attention mechanisms both for spatial and temporal aggregation. This allows ASTGCN to outperform state-of-the-art baselines on real-world datasets from the Caltrans performance measurement system.

Reinforcement Learning

Reinforcement Learning (RL) consists in designing an agent capable of learning through trial and error by interacting with an environment. This section only aims to briefly describe Markov Decision Processes (MDP) (Bellman 1957) and RL concepts. We refer the reader to (Sutton and Barto 2018) for a complete introduction to RL. We temporarily use the following notations here, not to be confused with notations from the previous section:

- Set S: a set of states.
- Set A: a set of actions.
- Set P: a set of transition probabilities. The probability $P(s'|s, a) = P_a(s, s')$ refers to the probability of tran-

sitioning from state $s \in S$ to state $s' \in S$ after taking action $a \in A$.

- Function R: a reward function. The transition from state $s \in S$ to state $s' \in S$ after taking action $a \in A$ results in an immediate reward $R(s'|s,a) = R_a(s,s')$.

In MDPs, the environment is fully observable and actions are instantaneous and non-deterministic. Nevertheless $\forall(s,a) \in (S,A),\ \exists! P_a(s,s')$. In other words, after taking action $a \in A$ in the state $s \in S$, a given set of probabilities exist for each state $s' \in S$ which defines the likelihood of transitioning into those states. Moreover, an *immediate reward* function $R_a(s,s')$ defines a given reward obtained from transitioning to a state $s' \in S$ after taking the action $a \in A$ in state $s \in S$. The aim for the agent is to devise an optimal *policy* π^* which specifies which action $\pi^*(s) \in A$ to take in any state s in order to maximize the total cumulative reward.

RL can generally be formulated as a 4-tuple (S, A, P, R), representing an agent interacting with the environment in a MDP. The agent interacts with the environment by following a policy π, and the goal is to find an optimal policy π^* by trial and error. Two main RL approaches exist: policy gradient optimization and value function optimization. In policy gradient approaches, a parameterizable function f_θ (θ being parameters) is used to approximate π directly. Through interaction with the environment, the agent is able to learn, given its current policy f_θ, which actions are more suited for given states in S. Thus, the agent can modify its policy f_θ to prioritize these actions. A popular choice for the function f_θ is neural networks, whose number of layers can be chosen according to the assumed complexity of the function approximated. On the other hand, value function optimization learns two different value functions Q and V, which define the policy π to follow. The state value function V is defined by: $V^\pi(s) = \mathbb{E}_\pi(\sum_{i=0}^{\infty} \gamma^i r_{i+1} | s_t = s)$, where s_t refers to the state of the agent at the current time step, $s_{t+1}, s_{t+2}, ...$ at future time steps; r_{i+1} refers to the immediate reward received by the agent at time step $t+i+1$; variable $\gamma \in]0,1]$ is a discount factor and $\mathbb{E}$ denotes the expectation. Intuitively, $V^\pi(s)$ corresponds to the expected sum of rewards when starting in state (s) and following policy π. The action value function is defined by $Q^\pi(s,a) = \mathbb{E}_\pi(\sum_{i=0}^{\infty} \gamma^i r_{i+1} | s_t = s, a_t = a)$ where a_t refers to the action taken by the agent at the current starting time step. Intuitively, it corresponds to the expected sum of rewards when starting in state (s), taking action a and following policy π afterwards. In environments with large and continuous state spaces, these functions are usually approximated using neural networks.

RL methods also belong to two categories: model-based and model-free. Model-based assumes knowledge of the transition probabilities in the MDP environment, while model-free does not. A popular approach for model-based is value iteration, which consists in updating Q-values by taking into account transition probabilities and known knowledge about transition states. Q-learning (Watkins and Dayan 1992) is a popular approach for model-free approaches. It follows the idea of 'pulling' a given Q-value toward the result obtained from a simulation with the environment every time with a learning rate, so as to approximate transition probabilities indirectly. Q-learning follows this update scheme:

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha[r_t + \gamma \max_{a_i} Q(s_{t+1},a_i) - Q(s_t,a_t)]$$

where α is the learning rate; $\delta_t = r_t + \gamma \max_{a_i} Q(s_{t+1},a_i) - Q(s_t,a_t)$ is called the temporal difference.

The use of Deep Q Neural Networks (DQN) (Mnih et al. 2013, 2015) has allowed RL tasks to achieve human level gameplay on games from the Atari 2600 plaform. More recently, AlphaGo (Silver et al. 2016), an AI program conceived to play the game of Go combining deep CNNs and Monte Carlo Tree Search (MCTS), managed to defeat the world champion of Go. AlphaGo uses supervised learning to learn from expert gameplay, and then refines the learned policy with RL (policy gradient) by playing against itself. AlphaGoZero (Silver et al. 2017), a newer version, is only trained with RL and achieves superior gaming performance than AlphaGo.

Model-based RL algorithms are usually more efficient than model-free ones since they can leverage planning using known environment dynamics. A solution for model-free approaches would be to learn the dynamics from interactions with the environment. Although learning dynamics which are accurate enough for planning has remained a challenge in model-free approaches, the PlaNet approach from (Hafner et al. 2019) achieves a breakthrough on this subject for image-based domains. PlaNet learns the dynamics model by relying on a sequence of latent states generated by an encoder-decoder architecture, rather than images directly. PlaNet chooses actions purely by planning in this latent space, this allows it to require far lower interaction with the environment to optimize its policy than previous recent approaches in model-free RL.

Lastly, in some situations, the environment may not be fully observable by the agent. This is the case in Partially Observable Markov Decision Processes (POMDP). Agents get sensory information and derive a probability distribution of states they may likely be in, and need to adapt their policy accordingly. Some popular works dealing with planning in POMDP include (Kurniawati, Hsu, and Lee 2008) who introduce a point-based POMDP algorithm for motion-planning, (Silver and Veness 2010) who propose an MCTS algorithm for planning in large POMDPs, (Somani et al. 2013) who present a random scenario sampling scheme to alleviate computational limitations and (Zhu et al. 2017) who propose a Deep Recurrent Q-Network to adapt RL tasks in POMDPs.

Path-Planning and Neural Networks

A*-based algorithms described previously are fast on small planning domains, but take exponentially longer as domain size and complexity grows. Probabilistic approaches such as PRMs and RRTs on the other hand construct a new graph with random sampling to bypass this complexity, but to guarantee consistent solution quality the sampling would need to be exponential again (LaValle, Branicky, and Lindemann 2004). Therefore, the idea of using neural networks

for path-planning has long been a problem of interest, although recent advances in machine learning has made it a viable option only recently. We explore a few such works.

(Glasius, Komoda, and Gielen 1995) is an early work which specifies obstacles into a topologically ordered neural map, and uses a neural network to trace a shortest path. The minimum of a Lyapunov function is used for convergence for neural activity. (Chen, Karl, and Van Der Smagt 2016), a more recent work, relies on *Deep Variational Bayes Filtering* (DVBF) (Karl et al. 2016) to embed dynamic movement primitives of a high dimensional humanoid movements in the latent space of a low dimensional variational autoencoder framework. RL has also been used for such purposes. (Levine and Koltun 2013) present a guided policy search algorithm that uses trajectory optimization to direct policy learning and avoid poor local optima, where policies are approximated by neural networks. This method is successfully applied to learn policies for planar swimming, hopping, walking and simulated 3D humanoid running. (Tamar et al. 2016) introduce a neural network to approximate the value iteration algorithm in order to predict outcomes that involve planning-based reasoning. Their use of CNNs limits their approach to path-planning on 2D grids and not motion planning in general.

Some *imitation learning*-based approaches have also been proposed. Imitation learning consists in having an expert provide demonstrations, in this case of desired trajectories. A neural network can then be used to approximate the behavior of the expert, and hopefully generalize outside of the scope of provided demonstrations. Imitation learning has been successful in several areas involving complex dynamical systems (Abbeel, Coates, and Ng 2010; Calinon et al. 2010). OracleNet, an extension of imitation learning for path-planning has been proposed recently in (Bency, Qureshi, and Yip 2019). OracleNet relies on an LSTM to build end-to-end trajectories in an iterative manner. The LSTM needs to be trained on optimal trajectories that span the entire configuration space of the considered environment before being used. Those optimal trajectories can be computed by algorithms such as A*. Although the proposed approach can be problematic if the framework needs to be quickly used in a newly known environment and no training time is available, OracleNet achieves performance which makes up for it. Paths are generated extremely fast, scaling almost linearly with dimensions reportedly. On a benchmark comprised of a point-mass robot with multiple degrees of freedom, OracleNet is compared to A* and RRT*. It achieves solution quality reportedly rivaling A* and far above RRT*, while its execution time remains far below the other two. In the context of path-planning under constraints, Osanlou *et al.* have combined a GNN with a constraint programming solver and a branch & bound tree search algorithm, observing in each case a significant improvement in the computation of solution paths, outperforming A*-based domain-tailored heuristics (Osanlou et al. 2021b, 2019, 2021c).

Temporal Planning With Uncertainty

Scheduling in the presence of uncertainty is an area of interest in artificial intelligence. In this section, we present necessary notions and work leading up to the Disjunctive Temporal Network with Uncertainty (DTNU).

Temporal Networks (Dechter, Meiri, and Pearl 1991) are a common formalism to represent temporal constraints over a set of timepoints (*e.g.* start/end of activities in a scheduling problem). A Simple Temporal Network (STN) Γ is defined by a pair:

$$\Gamma = (A, C)$$

Where:

- $A = (a_1, a_2, ..., a_n) \in \mathbb{R}^n$ is a set of n real controllable timepoint variables.
- C is a set of *free* constraints, each of which of the form: $a_j - a_i \in [x_k, y_k]$, where $a_i, a_j \in A$; $x_k \in \{-\infty\} \cup \mathbb{R}$; $y_k \in \mathbb{R} \cup \{+\infty\}$.

A solution to STN Γ is a complete set of assignments in $\mathbb{R}$ for each $a_i \in A$ which satisfies all constraints in C.

The Simple Temporal Networks with Uncertainty (STNUs) (Tsamardinos 2002; Vidal 1999) explicitly incorporate qualitative uncertainty into temporal networks. In STNUs, some events are *uncontrollable*. The only controllable aspect is when they start: how long they take to complete, however, is not known. Although the duration for completion is uncertain, it is often known to be within some bounds. These uncontrollable events are represented by a *contingency link*, *i.e.* a triplet $(a, [x, y], u)$, where a is a controllable timepoint (representing the start of the uncontrollable event), $[x, y]$ is the bounded duration of the uncontrollable event and u is an uncontrollable timepoint which signifies the end of the uncontrollable event. Uncontrollable timepoint u will occur on its own, at earliest x units of time after execution of a, y at latest.

Formally, an STNU Γ is defined as :

$$\Gamma = (A, U, C, L)$$

Where:

- $A = (a_1, a_2, ..., a_n) \in \mathbb{R}^n$ is a set of real controllable timepoint variables, which can be scheduled at any moment in time.
- $U = (u_1, u_2, ..., u_q) \in \mathbb{R}^q$ is a set of uncontrollable timepoint variables.
- Each uncontrollable timepoint $u_j \in U$ is linked to exactly one controllable timepoint $a_i \in A$ by a contingency link $l \in L : l = (a_i, [x, y], u_j)$
- C is a set of free constraints of the same form as with STNs, except constraints can also involve uncontrollable timepoints in addition to controllable timepoints.

We refer to timepoints in general (controllable or uncontrollable) as $V = A \cup U$. Different types of *controllability* exist (Vidal 1999):

- *Strong Controllablity* (SC): An STNU $\Gamma = (A, U, C, L)$ is strongly controllable if there exists at least one universal schedule of controllable timepoints $\{a_1 = w_1, a_2 =$

$w_2, ..., a_n = w_n\}$ which satisfies the constraints in C regardless of the values taken by uncontrollable timepoints U.

- *Weak Controllablity* (WC): An STNU $\Gamma = (A, U, C, L)$ is weakly controllable if, for every value outcome of uncontrollable timepoints U, there is at least one schedule of controllable timepoints $\{a_1 = w_1, a_2 = w_2, ..., a_n = w_n\}$ which satisfies the constraints in C.
- *Dynamic Controllablity* (DC): An STNU $\Gamma = (A, U, C, L)$ is dynamically controllable if there is a reactive strategy which guarantees constraints in C will be satisfied if the scheduling strategy is followed by a controller agent, while observing possible occurrences of uncontrollable timepoints and using this knowledge to adapt decisions. It is said the problem is DC if and only if it admits a valid dynamic strategy expressed as a map from partial schedules to Real-Time Execution Decisions (RTEDs) (Cimatti, Micheli, and Roveri 2016). A partial schedule represents the current scheduling state, *i.e.* the set of timepoints that have been scheduled so far and their timing. RTEDs are popular semantics used to express a DC strategy (Hunsberger 2009). RTEDs regroup two possible actions: **(1)** The wait action, *i.e.* wait for an uncontrollable timepoint to occur. **(2)** The $(t, \mathcal{X})$ action, *i.e.* if nothing happens before time $t \in \mathbb{R}$, schedule the controllable timepoints in $\mathcal{X}$ at t. A strategy is valid if, for every possible occurrence of the uncontrollable timepoints, controllable timepoints get scheduled in a way that all free constraints are satisfied.

Considerable work has resulted in algorithms to determine whether or not an STNU is DC or not (Morris and Muscettola 2005; Morris 2014), leading to $\mathcal{O}(N^3)$ worst-case DC-checking algorithms, where N is the number of timepoints of the STNU. These DC-checking algorithms also synthesize valid DC strategies executable in $\mathcal{O}(N^3)$ (Hunsberger 2016; Morris 2014). Disjunctive Temporal Networks with Uncertainty (DTNUs) generalize STNUs by allowing the presence of disjunctions in the constraints C or contingency links L. Formally, each constraint in C is of the form : $\vee_{k=1}^{q} v_{k,j} - v_{k,i} \in [x_k, y_k]$. Furthermore, each contingency link $l \in L$ is of the form : $(a_i, \vee_{k=1}^{q'}[x_k, y_k], u_j)$ where $x_k \leq y_k \leq x_{k+1} \leq y_{k+1}\ \forall k = 1, 2, ..., q' - 1$. All controllability types for STNUs remain available for DTNUs. The introduction of disjunctions inside C and L renders STNU's $\mathcal{O}(N^3)$ DC-checking algorithms unavailable for DTNUs. In fact, the complexity of DC checking for DTNUs is $PSPACE$-complete (Bhargava and Williams 2019), making this a highly challenging problem. The difficulty in proving or disproving DC arises from the need to check all possible combinations of disjuncts in order to handle all possible occurrence outcomes of the uncontrollable timepoints. The only known approach for DC-checking and DC strategy generation for DTNUs is based on expressing DTNUs as timed-game automata (TGAs) (Cimatti et al. 2014). TGAs can then be solved by the UPPAAL-TIGA software (Behrmann et al. 2007). In (Cimatti, Micheli, and Roveri 2016), authors express DTNUs as TGAs in the same way, but use a pruning procedure based on satisfiability modulo theory and achieve superior results than with UPPAAL-TIGA. Authors in (Osanlou et al. 2021a, 2022) design a tree search algorithm that searches in *Restricted Time-based Dynamic Controllability* (R-TDC), a subspace of DC. They show R-TDC allows higher strategy search efficiency than TGA-based approaches while retaining very high DC coverage, thus almost always finding a strategy when a DC one exists on considered benchmarks. They also note a significant increase in search performance for harder problems owing to a heuristic based on graph neural network they use for search guidance.

References

Abbeel, P.; Coates, A.; and Ng, A. Y. 2010. Autonomous helicopter aerobatics through apprenticeship learning. *The International Journal of Robotics Research*, 29(13): 1608–1639.

Adiyatov, O.; and Varol, H. A. 2013. Rapidly-exploring random tree based memory efficient motion planning. In *2013 IEEE international conference on mechatronics and automation*, 354–359. IEEE.

Atwood, J.; and Towsley, D. 2016. Diffusion-convolutional neural networks. In *Advances in neural information processing systems*, 1993–2001.

Badue, C.; Guidolini, R.; Carneiro, R. V.; Azevedo, P.; Cardoso, V. B.; Forechi, A.; Jesus, L.; Berriel, R.; Paixão, T. M.; Mutz, F.; et al. 2020. Self-driving cars: A survey. *Expert Systems with Applications*, 113816.

Behrmann, G.; Cougnard, A.; David, A.; Fleury, E.; Larsen, K. G.; and Lime, D. 2007. Uppaal-tiga: Time for playing games! In *International Conference on Computer Aided Verification*, 121–125. Springer.

Bellman, R. 1957. A Markovian decision process. *Journal of mathematics and mechanics*, 679–684.

Bellman, R. 1966. Dynamic programming. *Science*, 153(3731): 34–37.

Bency, M. J.; Qureshi, A. H.; and Yip, M. C. 2019. Neural path planning: Fixed time, near-optimal path generation via oracle imitation. *arXiv preprint arXiv:1904.11102*.

Bhargava, N.; and Williams, B. C. 2019. Complexity bounds for the controllability of temporal networks with conditions, disjunctions, and uncertainty. *Artificial Intelligence*, 271: 1–17.

Botea, A.; Müller, M.; and Schaeffer, J. 2004. Near Optimal Hierarchical Path-Finding. *J. Game Dev.*, 1(1): 1–30.

Bruna, J.; Zaremba, W.; Szlam, A.; and Lecun, Y. 2014. Spectral networks and locally connected networks on graphs. In *Proceedings of the International Conference on Learning Representations (ICLR 2014)*.

Calinon, S.; D'halluin, F.; Sauser, E. L.; Caldwell, D. G.; and Billard, A. G. 2010. Learning and reproduction of gestures by imitation. *IEEE Robotics & Automation Magazine*, 17(2): 44–54.

Chen, N.; Karl, M.; and Van Der Smagt, P. 2016. Dynamic movement primitives in latent space of time-dependent variational autoencoders. In *2016 IEEE-RAS 16th International Conference on Humanoid Robots (Humanoids)*, 629–636. IEEE.

Cho, K.; Van Merriënboer, B.; Gulcehre, C.; Bahdanau, D.; Bougares, F.; Schwenk, H.; and Bengio, Y. 2014. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.

Cimatti, A.; Hunsberger, L.; Micheli, A.; Posenato, R.; and Roveri, M. 2014. Sound and complete algorithms for checking the dynamic controllability of temporal networks with uncertainty, disjunction and observation. In *2014 21st International Symposium on Temporal Representation and Reasoning*, 27–36. IEEE.

Cimatti, A.; Micheli, A.; and Roveri, M. 2016. Dynamic controllability of disjunctive temporal networks: Validation and synthesis of executable strategies. In *Thirtieth AAAI Conference on Artificial Intelligence*.

Collins, A. D.; Agarwal, P. K.; and Harer, J. L. 2003. HPRM: a hierarchical PRM. In *2003 IEEE International Conference on Robotics and Automation (Cat. No. 03CH37422)*, volume 3, 4433–4438. IEEE.

Dechter, R.; Meiri, I.; and Pearl, J. 1991. Temporal constraint networks. *Artificial intelligence*, 49(1-3): 61–95.

Defferrard, M.; Bresson, X.; and Vandergheynst, P. 2016. Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in neural information processing systems*, 3844–3852.

Ferguson, D.; and Stentz, A. 2005. The delayed D* algorithm for efficient path replanning. In *Proceedings of the 2005 IEEE international conference on robotics and automation*, 2045–2050. IEEE.

Ferguson, D.; and Stentz, A. 2007. Field D*: An interpolation-based path planner and replanner. In *Robotics research*, 239–253. Springer.

Fikes, R. E.; and Nilsson, N. J. 1971. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial intelligence*, 2(3-4): 189–208.

Fukushima, K.; and Miyake, S. 1982. Neocognitron: A self-organizing neural network model for a mechanism of visual pattern recognition. In *Competition and cooperation in neural nets*, 267–285. Springer.

Funahashi, K.-I. 1989. On the approximate realization of continuous mappings by neural networks. *Neural networks*, 2(3): 183–192.

Gammell, J. D.; Srinivasa, S. S.; and Barfoot, T. D. 2014. Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic. In *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2997–3004. IEEE.

Gilmer, J.; Schoenholz, S. S.; Riley, P. F.; Vinyals, O.; and Dahl, G. E. 2017. Neural Message Passing for Quantum Chemistry. In *International Conference on Machine Learning*, 1263–1272.

Glasius, R.; Komoda, A.; and Gielen, S. C. 1995. Neural network dynamics for path planning and obstacle avoidance. *Neural Networks*, 8(1): 125–133.

Guettier, C.; and Lucas, F. 2016. A constraint-based approach for planning unmanned aerial vehicle activities. *The Knowledge Engineering Review*, 31(5): 486.

Guo, S.; Lin, Y.; Feng, N.; Song, C.; and Wan, H. 2019. Attention based spatial-temporal graph convolutional networks for traffic flow forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, 922–929.

Hafner, D.; Lillicrap, T.; Fischer, I.; Villegas, R.; Ha, D.; Lee, H.; and Davidson, J. 2019. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning*, 2555–2565. PMLR.

Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. In *Advances in neural information processing systems*, 1024–1034.

Hansen, E. A.; and Zhou, R. 2007. Anytime heuristic search. *Journal of Artificial Intelligence Research*, 28: 267–297.

Harabor, D.; and Botea, A. 2008. Hierarchical path planning for multi-size agents in heterogeneous environments. In *2008 IEEE Symposium On Computational Intelligence and Games*, 258–265. IEEE.

Harabor, D. D.; Grastien, A.; Öz, D.; and Aksakalli, V. 2016. Optimal any-angle pathfinding in practice. *Journal of Artificial Intelligence Research*, 56: 89–118.

Hart, P. E.; Nilsson, N. J.; and Raphael, B. 1968. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2): 100–107.

Hunsberger, L. 2009. Fixing the semantics for dynamic controllability and providing a more practical characterization of dynamic execution strategies. In *2009 16th International Symposium on Temporal Representation and Reasoning*, 155–162. IEEE.

Hunsberger, L. 2016. Efficient execution of dynamically controllable simple temporal networks with uncertainty. *Acta Informatica*, 53(2): 89–147.

Ichter, B.; Schmerling, E.; Lee, T.-W. E.; and Faust, A. 2020. Learned critical probabilistic roadmaps for robotic motion planning. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 9535–9541. IEEE.

Jain, A.; Zamir, A. R.; Savarese, S.; and Saxena, A. 2016. Structural-rnn: Deep learning on spatio-temporal graphs. In *Proceedings of the ieee conference on computer vision and pattern recognition*, 5308–5317.

Kannan, A.; Gupta, P.; Tiwari, R.; Prasad, S.; Khatri, A.; and Kala, R. 2016. Robot motion planning using adaptive hybrid sampling in probabilistic roadmaps. *Electronics*, 5(2): 16.

Karaman, S.; and Frazzoli, E. 2010. Incremental sampling-based algorithms for optimal motion planning. *Robotics Science and Systems VI*, 104(2).

Karl, M.; Soelch, M.; Bayer, J.; and Van der Smagt, P. 2016. Deep variational bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint arXiv:1605.06432*.

Kavraki, L. E.; Svestka, P.; Latombe, J.-C.; and Overmars, M. H. 1996. Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE transactions on Robotics and Automation*, 12(4): 566–580.

Kipf, T. 2021. A GCN architecture.

Kipf, T. N.; and Welling, M. 2017. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*.

Koenig, S.; and Likhachev, M. 2002a. Improved fast replanning for robot navigation in unknown terrain. In *Proceedings 2002 IEEE International Conference on Robotics and Automation (Cat. No. 02CH37292)*, volume 1, 968–975. IEEE.

Koenig, S.; and Likhachev, M. 2002b. Incremental a. In *Advances in neural information processing systems*, 1539–1546.

Kumar, A. 2020. Robot Manipulator Motion Planning Visualization.

Kurniawati, H.; Hsu, D.; and Lee, W. S. 2008. Sarsop: Efficient point-based pomdp planning by approximating optimally reachable belief spaces. In *Robotics: Science and systems*, volume 2008. Zurich, Switzerland.

Lai, T.; Ramos, F.; and Francis, G. 2019. Balancing global exploration and local-connectivity exploitation with rapidly-exploring random disjointed-trees. In *2019 International Conference on Robotics and Automation (ICRA)*, 5537–5543. IEEE.

Latombe, C. 2020. A Probabilistic roadmap.

LaValle, S. M. 1998. Rapidly-exploring random trees: A new tool for path planning.

LaValle, S. M.; Branicky, M. S.; and Lindemann, S. R. 2004. On the relationship between classical grid search and probabilistic roadmaps. *The International Journal of Robotics Research*, 23(7-8): 673–692.

LeCun, Y.; Bengio, Y.; et al. 1995. Convolutional networks for images, speech, and time series. *The handbook of brain theory and neural networks*, 3361(10): 1995.

Levine, S.; and Koltun, V. 2013. Guided policy search. In *International Conference on Machine Learning*, 1–9.

Li, R.; and Wang, S. 2018. Adaptive Graph Convolutional Neural Networks. In *AAAI Conference on Artificial Intelligence*.

Li, Y.; Tarlow, D.; Brockschmidt, M.; and Zemel, R. 2015. Gated graph sequence neural networks. *arXiv preprint arXiv:1511.05493*.

Li, Y.; Yu, R.; Shahabi, C.; and Liu, Y. 2017. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. *arXiv preprint arXiv:1707.01926*.

Likhachev, M.; Ferguson, D. I.; Gordon, G. J.; Stentz, A.; and Thrun, S. 2005. Anytime Dynamic A*: An Anytime, Replanning Algorithm. In *ICAPS*, volume 5, 262–271.

Likhachev, M.; Gordon, G. J.; and Thrun, S. 2003. ARA*: Anytime A* with provable bounds on sub-optimality. *Advances in neural information processing systems*, 16: 767–774.

McDermott, D.; Ghallab, M.; Howe, A.; Knoblock, C.; Ram, A.; Veloso, M.; Weld, D.; and Wilkins, D. 1998. PDDL-the planning domain definition language.

Micheli, A. 2009. Neural network for graphs: A contextual constructive approach. *IEEE Transactions on Neural Networks*, 20(3): 498–511.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Graves, A.; Antonoglou, I.; Wierstra, D.; and Riedmiller, M. 2013. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. 2015. Human-level control through deep reinforcement learning. *nature*, 518(7540): 529–533.

Morris, P. 2014. Dynamic controllability and dispatchability relationships. In *International Conference on AI and OR Techniques in Constriant Programming for Combinatorial Optimization Problems*, 464–479. Springer.

Morris, P. H.; and Muscettola, N. 2005. Temporal dynamic controllability revisited. In *Aaai*, 1193–1198.

Muscettola, N.; Nayak, P. P.; Pell, B.; and Williams, B. C. 1998. Remote agent: To boldly go where no AI system has gone before. *Artificial intelligence*, 103(1-2): 5–47.

Nash, A.; Daniel, K.; Koenig, S.; and Felner, A. 2007. Theta^*: Any-angle path planning on grids. In *AAAI*, volume 7, 1177–1183.

Nilsson, N. J. 1984. Shakey the robot. Technical report, SRI INTERNATIONAL MENLO PARK CA.

Osanlou, K.; Bursuc, A.; Guettier, C.; Cazenave, T.; and Jacopin, E. 2019. Optimal Solving of Constrained Path-Planning Problems with Graph Convolutional Networks and Optimized Tree Search. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3519–3525. IEEE.

Osanlou, K.; Frank, J.; Benton, J.; Bursuc, A.; Guettier, C.; Cazenave, T.; and Jacopin, E. 2022. Solving Disjunctive Temporal Networks with Uncertainty under Restricted Time-Based Controllability using Tree Search and Graph Neural Networks. In *Thirty-sixth AAAI conference on artificial intelligence*.

Osanlou, K.; Frank, J.; Benton, J.; Bursuc, A.; Guettier, C.; Jacopin, E.; and Cazenave, T. 2021a. Time-based Dynamic Controllability of Disjunctive Temporal Networks with Uncertainty: A Tree Search Approach with Graph Neural Network Guidance. *arXiv preprint arXiv:2108.01068*.

Osanlou, K.; Guettier, C.; Bursuc, A.; Cazenave, T.; and Jacopin, E. 2021b. Constrained shortest path search with graph convolutional neural networks. *arXiv preprint arXiv:2108.00978*.

Osanlou, K.; Guettier, C.; Bursuc, A.; Cazenave, T.; and Jacopin, E. 2021c. Learning-based preference prediction for constrained multi-criteria path-planning. *arXiv preprint arXiv:2108.01080*.

Przybylski, M.; and Putz, B. 2017. D* Extra Lite: A Dynamic A* With Search–Tree Cutting and Frontier–Gap Repairing. *International Journal of Applied Mathematics and Computer Science*, 27(2): 273–290.

Rabideau, G.; and Benowitz, E. 2017. Prototyping an onboard scheduler for the mars 2020 rover.

Scarselli, F.; Gori, M.; Tsoi, A. C.; Hagenbuchner, M.; and Monfardini, G. 2008. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1): 61–80.

Seo, Y.; Defferrard, M.; Vandergheynst, P.; and Bresson, X. 2018. Structured sequence modeling with graph convolutional recurrent networks. In *International Conference on Neural Information Processing*, 362–373. Springer.

Silver, D.; Huang, A.; Maddison, C. J.; Guez, A.; Sifre, L.; Van Den Driessche, G.; Schrittwieser, J.; Antonoglou, I.; Panneershelvam, V.; Lanctot, M.; et al. 2016. Mastering the game of Go with deep neural networks and tree search. *nature*, 529(7587): 484–489.

Silver, D.; Schrittwieser, J.; Simonyan, K.; Antonoglou, I.; Huang, A.; Guez, A.; Hubert, T.; Baker, L.; Lai, M.; Bolton, A.; et al. 2017. Mastering the game of go without human knowledge. *Nature*, 550(7676): 354.

Silver, D.; and Veness, J. 2010. Monte-Carlo planning in large POMDPs. In *Advances in neural information processing systems*, 2164–2172.

Somani, A.; Ye, N.; Hsu, D.; and Lee, W. S. 2013. DESPOT: Online POMDP planning with regularization. In *Advances in neural information processing systems*, 1772–1780.

Sperduti, A.; and Starita, A. 1997. Supervised neural networks for the classification of structures. *IEEE Transactions on Neural Networks*, 8(3): 714–735.

Stentz, A.; et al. 1995. The focussed dˆ* algorithm for real-time replanning. In *IJCAI*, volume 95, 1652–1659.

Sturtevant, N.; and Buro, M. 2005. Partial pathfinding using map abstraction and refinement. In *AAAI*, volume 5, 1392–1397.

Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement learning: An introduction*. MIT press.

Tamar, A.; Wu, Y.; Thomas, G.; Levine, S.; and Abbeel, P. 2016. Value iteration networks. In *Advances in Neural Information Processing Systems*, 2154–2162.

Tsamardinos, I. 2002. A probabilistic approach to robust execution of temporal plans with uncertainty. In *Hellenic Conference on Artificial Intelligence*, 97–108. Springer.

Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.

Vidal, T. 1999. Handling contingency in temporal constraint networks: from consistency to controllabilities. *Journal of Experimental & Theoretical Artificial Intelligence*, 11(1): 23–45.

Watkins, C. J.; and Dayan, P. 1992. Q-learning. *Machine learning*, 8(3-4): 279–292.

Wu, Z.; Pan, S.; Long, G.; Jiang, J.; and Zhang, C. 2019. Graph wavenet for deep spatial-temporal graph modeling. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, 1907–1913. AAAI Press.

Zhu, P.; Li, X.; Poupart, P.; and Miao, G. 2017. On improving deep reinforcement learning for pomdps. *arXiv preprint:1704.07978*.

View publication stats